

Comprehensive Image Enhancement and Restoration Analysis

Matthew Harry Indriadi - 24/532723/PA/22527

September 27, 2026

Abstract

This report documents a comprehensive image processing and enhancement project focused on improving low-light and degraded image quality. The work implements multiple state-of-the-art algorithms including histogram equalization, contrast-limited adaptive histogram equalization (CLAHE), gamma correction, unsharp masking, denoising, and deblurring techniques. A dark JPEG image (576 x 1024 pixels) is systematically processed through various enhancement pipelines, with detailed comparisons of results. The project demonstrates practical applications of fundamental computer vision principles using OpenCV, NumPy, and SciPy libraries.

1 Introduction

Image enhancement is a fundamental task in digital image processing with applications in medical imaging, surveillance, astronomy, and general photography. This project explores multiple enhancement techniques to improve the visibility and quality of a dark image. The goal is to recover details obscured by poor lighting conditions while maintaining image fidelity.

1.1 Objectives

- Apply and compare multiple brightness enhancement algorithms
- Implement advanced contrast adjustment techniques
- Reduce image noise while preserving details
- Sharpen edges and enhance fine structures
- Evaluate quality metrics and visual results
- Create a modular, reusable enhancement pipeline

2 Technical Framework

2.1 Libraries and Tools

- **OpenCV 5.0.0:** Core computer vision algorithms and image I/O
- **NumPy 2.1.3:** Numerical operations and array manipulation
- **Matplotlib:** Visualization and histogram analysis
- **scikit-image:** Advanced image processing utilities
- **SciPy:** Scientific computing and filtering operations

2.2 Input Data

- **Filename:** dark_image.jpeg
- **Original Resolution:** 2160 x 3840 pixels
- **Working Resolution:** 576 x 1024 pixels (downscaled for efficiency)
- **Color Space:** RGB (converted from BGR)
- **Data Type:** uint8 (8-bit unsigned integer)

3 Enhancement Algorithms

3.1 1. Histogram Equalization

3.1.1 Theory

Histogram equalization redistributes pixel intensities to achieve a more uniform distribution across the intensity range. This improves contrast and visibility of details.

3.1.2 Mathematical Formulation

For a grayscale image, the cumulative distribution function (CDF) is:

$$CDF(i) = \frac{n_i}{N} \quad (1)$$

where n_i is the number of pixels with intensity i and N is the total number of pixels. The enhanced intensity is:

$$I'(x, y) = \lfloor (L - 1) \cdot CDF(I(x, y)) \rfloor \quad (2)$$

3.1.3 Implementation

For color images, the algorithm is applied in the YCrCb color space to preserve color information:

```
def histogram_equalization(image):  
    ycrb = cv2.cvtColor(image, cv2.COLOR_RGB2YCrCb)  
    ycrb[:, :, 0] = cv2.equalizeHist(ycrb[:, :, 0])  
    return cv2.cvtColor(ycrb, cv2.COLOR_YCrCb2RGB)
```

3.2 2. Contrast Limited Adaptive Histogram Equalization (CLAHE)

3.2.1 Theory

CLAHE addresses over-enhancement artifacts from global histogram equalization by applying local histogram equalization with a clipping limit. The image is divided into tiles, and histograms are computed independently per tile.

3.2.2 Parameters

- **Clip Limit:** Threshold for histogram clipping (typical range: 1.0~4.0)
- **Tile Grid Size:** Division of image into tiles (typical: 8~8 or 16~16)

3.2.3 Implementation

```
def clahe_enhance(image, clip_limit=2.0, tile_grid=(8, 8)):
    clahe = cv2.createCLAHE(clipLimit=clip_limit,
                             tileGridSize=tile_grid)
    lab = cv2.cvtColor(image, cv2.COLOR_RGB2LAB)
    lab[:, :, 0] = clahe.apply(lab[:, :, 0])
    return cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)
```

3.3 3. Gamma Correction

3.3.1 Theory

Gamma correction applies a power-law transformation to adjust image brightness and contrast. This is particularly effective for correcting non-linear sensor responses.

3.3.2 Mathematical Formulation

$$I'(x, y) = I(x, y)^{1/\gamma} \quad (3)$$

where:

- $\gamma < 1$: Brightening
- $\gamma > 1$: Darkening
- $\gamma = 1$: No change

3.3.3 Implementation

```
def gamma_correction(image, gamma=1.0):
    inv_gamma = 1.0 / gamma
    table = np.array([((i / 255.0) ** inv_gamma) * 255
                      for i in np.arange(0, 256)]).astype('uint8')
    return cv2.LUT(image, table)
```

3.4 4. Brightness and Contrast Adjustment

3.4.1 Theory

Linear scaling of pixel values: $I' = \alpha I + \beta$

- α (contrast multiplier): Controls contrast magnitude
- β (brightness offset): Adds uniform brightness offset

3.4.2 Implementation

```
def adjust_brightness_contrast(image, alpha=1.0, beta=0):
    return cv2.convertScaleAbs(image, alpha=alpha, beta=beta)
```

3.5 5. Sharpening

3.5.1 5.1 Unsharp Mask

The unsharp masking technique enhances edges by subtracting a blurred version from the original:

$$I_{sharp} = I_{original} + \lambda(I_{original} - I_{blurred}) \quad (4)$$

where λ is the sharpening strength.

```
def unsharp_mask(image, sigma=2.0, strength=1.5):
    blurred = cv2.GaussianBlur(image, (0, 0), sigma)
    sharpened = cv2.addWeighted(image, 1 + strength,
                                blurred, -strength, 0)
    return np.clip(sharpened, 0, 255).astype(np.uint8)
```

3.5.2 5.2 Laplacian Sharpening

A 3x3 sharpening kernel emphasizing center pixel:

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (5)$$

```
def sharpen_image(image):
    kernel = np.array([[ 0, -1,  0],
                       [-1,  5, -1],
                       [ 0, -1,  0]])
    return cv2.filter2D(image, -1, kernel)
```

3.6 6. Denoising

3.6.1 Non-Local Means Denoising

This algorithm leverages similarity between image patches to reduce noise while preserving structure:

$$I'(x) = \frac{\sum_y w(x, y) I(y)}{\sum_y w(x, y)} \quad (6)$$

where $w(x, y)$ is the patch similarity weight between positions x and y .

```
def denoise_image(image):
    if image.ndim == 3:
        return cv2.fastNlMeansDenoisingColored(image, None, 10, 10, 7,
                                                21)
    return cv2.fastNlMeansDenoising(image, None, 10, 7, 21)
```

3.7 7. Wiener Deblurring

3.7.1 Theory

Wiener deconvolution is applied in the frequency domain to reverse blur effects. The filter is:

$$W(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + \sigma_n^2} \quad (7)$$

where H is the point spread function (PSF), H^* is its conjugate, and σ_n^2 is the noise variance.

3.7.2 Implementation

```
def wiener_deblur(image, kernel_size=5, noise=0.01):
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY) if image.ndim == 3
    else image
    kernel = np.ones((kernel_size, kernel_size)) / (kernel_size ** 2)
    psf = np.zeros_like(gray, dtype=np.float32)
    kh, kw = kernel.shape
    psf[:kh, :kw] = kernel
    psf = np.roll(psf, -kh // 2, axis=0)
    psf = np.roll(psf, -kw // 2, axis=1)
    psf_fft = fft2(psf)
    gray_fft = fft2(gray.astype(np.float32))
    result = np.real(iff2(gray_fft * np.conj(psf_fft) / (np.abs(
        psf_fft) ** 2 + noise)))
    return np.clip(result, 0, 255).astype(np.uint8)
```

4 Enhancement Pipelines

4.1 Pipeline 1: Automatic Enhancement

Sequential application of multiple techniques for comprehensive improvement:

1. Histogram equalization (global contrast)
2. Gamma correction with $\gamma = 0.4$ (brightness boost)
3. Brightness/contrast adjustment ($\alpha = 1.2$, $\beta = 20$)
4. Unsharp masking ($\sigma = 2.0$, strength = 1.5)
5. Non-local means denoising

4.2 Pipeline 2: CLAHE-Based Enhancement

Focused on adaptive contrast without oversaturation:

1. CLAHE with clip limit 2.0 and 8x8 tiles
2. Unsharp masking for edge enhancement
3. Gamma correction for fine-tuning brightness

4.3 Pipeline 3: Denoising-First Approach

Prioritizes noise reduction before enhancement:

1. Non-local means denoising
2. CLAHE for contrast
3. Unsharp masking for sharpness

5 Results and Analysis

5.1 Output Images

The project generates four enhanced versions:

- **dark_image_enhanced.jpg**: Full automatic enhancement pipeline
- **dark_image_clahe.jpg**: CLAHE-based enhancement
- **dark_image_sharp.jpg**: Sharpness-focused processing
- **dark_image_denoised.jpg**: Denoising-first approach

5.2 Quality Assessment

5.2.1 Qualitative Metrics

- **Brightness**: Effective light recovery from dark areas
- **Contrast**: Visible distinction between objects
- **Sharpness**: Edge definition and detail clarity
- **Artifacts**: Over-enhancement, halos, or color distortion
- **Noise**: Visible grain or texture artifacts

5.2.2 Observations

- CLAHE provides balanced enhancement without extreme artifacts
- Gamma correction is effective for dark image recovery
- Unsharp masking improves perceived sharpness but can amplify noise
- Non-local means denoising preserves structure while reducing noise
- Combined pipelines yield superior results compared to single techniques

6 Color Space Considerations

6.1 RGB vs. YCrCb vs. LAB

Color Space	Use Case	Advantages
RGB	Display and general processing	Intuitive, standard format
YCrCb	Histogram equalization	Separates luminance from chrominance
LAB	CLAHE, color-independent processing	Perceptually uniform, independent color channels

Working in appropriate color spaces prevents color distortion and preserves perceptual quality during enhancement.

7 Performance Considerations

7.1 Computational Complexity

Algorithm	Complexity
Histogram Equalization	$O(n)$
CLAHE	$O(n)$ (local)
Gamma Correction	$O(n)$ (LUT-based)
Gaussian Blur	$O(n \log k)$, where k is kernel size
Non-Local Means	$O(n \sqrt{k})$ (patch-based)
Wiener Deconvolution	$O(n \log n)$ (FFT-based)

7.2 Memory Usage

The downscaling from 2160 x 3840 to 576 x 1024 reduces memory footprint by approximately 85 percent, enabling efficient processing on standard hardware.

8 Practical Recommendations

8.1 Algorithm Selection

For **dark images**:

- Start with gamma correction ($\gamma = 0.3$ – 0.5)
- Apply CLAHE for adaptive contrast
- Use unsharp masking for edge enhancement

For **noisy images**:

- Begin with non-local means denoising
- Apply CLAHE with moderate clipping
- Avoid aggressive sharpening

For **blurred images**:

- Use unsharp masking as primary technique
- Consider Wiener deconvolution for structured blur
- Adjust kernel size based on blur extent

8.2 Parameter Tuning

Algorithm	Parameter	Recommended Range
Gamma Correction	γ	0.3–0.7 (darkening), 1.2–2.0 (brightening)
CLAHE	Clip Limit	1.0–4.0
CLAHE	Tile Size	8 to 16
Unsharp Mask	σ	1.0–3.0
Unsharp Mask	Strength	0.5–2.0
Brightness/Contrast	α	1.0–1.5
Brightness/Contrast	β	-50 to 50

9 Limitations and Future Work

9.1 Current Limitations

- Histogram equalization can produce unnatural color shifts
- Wiener deconvolution requires known PSF (rarely available)
- Parameter tuning is often manual and domain-specific
- Single-image enhancement faces inherent information loss limits

9.2 Future Improvements

- Implement deep learning-based enhancement (e.g., autoencoders, GANs)
- Develop automatic parameter optimization using quality metrics
- Extend to video processing with temporal consistency constraints
- Incorporate blind deconvolution for unknown PSF
- Add machine learning-based artifact detection and removal

10 Conclusion

This project demonstrates a comprehensive approach to image enhancement through multiple algorithmic techniques. The modular implementation allows flexible combination of methods to address specific image degradation challenges. Key findings include:

1. CLAHE provides the most balanced enhancement for dark images
2. Multi-stage pipelines outperform single-technique approaches
3. Proper color space selection is critical for quality preservation
4. Non-local means denoising effectively reduces noise while maintaining detail
5. Gamma correction is highly effective for dark image recovery

The produced enhanced images show significant improvement in brightness, contrast, and perceived detail compared to the original dark image. The modular codebase enables rapid experimentation with different enhancement strategies and parameter values.

11 References

- Gonzalez, R. C., and Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
- Pizer, S. M., et al. (1987). Adaptive histogram equalization and its variations. *Computer Vision, Graphics, and Image Processing*, 39(3), 355-368.
- Buades, A., Coll, B., and Morel, J. M. (2005). A non-local algorithm for image denoising. *CVPR*, 2, 60-65.
- Bradski, G. (2000). The OpenCV library. *Dr. Dobbs Journal*, 25(11), 120-126.
- Harris, J. L. (1966). Diffraction and resolving power. *Journal of the Optical Society of America*, 54(7), 931-936.